PEMBELAJARAN MESIN (*MACHINE LEARNING*)

Praktikum 2 Pemahaman Data dan Pra Pengolahan Data

Dosen Pengampu Mata Kuliah

Dr. Ely Setyo Astuti, S.T., M.T.



Disusun Oleh :

Siti Nikmatus Sholihah 244107020014

PROGRAM STUDI SARJANA TERAPAN TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

2026

Praktikum 2a
Exploratory Data Analysis (EDA)

Langkah 0 - Persiapan Lingkungan

(Opsional) Lakukan instalasi library pandas, numpy, dan matplotlib. Pandas merupakan library untuk manipulasi data tabular. Numpy digunakan untuk manipulasi angka. Matplotlib digunakan untuk kebutuhan visualisasi data.

```
!pip install -q pandas matplotlib numpy
```
[1] ✓ 9.0s

Import library tersebut pada cell notebook

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```
[2] ✓ 17.8s

Langkah 1 - Memuat Data

Jika Anda menggunakan Google Colab, lakukan proses unggah data pada instance Google Colab terlebih dahulu.

Lakukan pemuatan data dengan perintah,

```
df = pd.read_csv('Titanic-Dataset.csv')
```
[4] ✓ 0.1s

Cek 5 data pertama dengan menggunakan perintah,

```
df.head()
```
[5] ✓ 0.0s

Maka akan muncul tampilan dibawah ini,

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

Jika Anda membutuhkan preview lebih banyak data, Anda dapat memberikan parameter jumlah. Contoh, untuk menampilkan 100 data pertama, gunakan,

```
df.head(100)
```
[6] ✓ 0.0s Python

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
...	...	...	...	...	...	...	...	...	...	...	...	...
95	96	0	3	Shorney, Mr. Charles Joseph	male	NaN	0	0	374910	8.0500	NaN	S
96	97	0	1	Goldschmidt, Mr. George B	male	71.0	0	0	PC 17754	34.6542	A5	C
97	98	1	1	Greenfield, Mr. William Bertram	male	23.0	0	1	PC 17759	63.3583	D10 D12	C
98	99	1	2	Doling, Mrs. John T (Ada Julia Bone)	female	34.0	0	1	231919	23.0000	NaN	S
99	100	0	2	Kantor, Mr. Sinai	male	34.0	1	0	244367	26.0000	NaN	S

100 rows × 12 columns

Langkah 2 - Inspeksi Data

Pada tahap ini, kita lakukan pemahaman terhadap data berdasarkan nilai-nilai stastik dan kondisi data. Beberapa hal yang biasa dilakukan diantara adalah,

1. Pengecekan dimensi → Untuk mengetahui jumlah baris dan kolom dari data. Pada data tabular, cara ini berguna untuk mendapatkan informasi jumlah data dan variabel.
2. Pengecekan tipe data → Untuk mengetahui tipe data setiap variabel. Bermanfaat untuk proses pengolahan selanjutnya
3. Pengecekan nilai statistik deskriptif → Untuk mengetahui ukuran kecenderuangan memusat atau menyebarnya suatu data.

Pengecekan Dimensi

```
df.shape
```
[8] ✓ 0.0s

Anda akan mendapatkan hasil,

```
df.shape
```
[8] ✓ 0.0s

```
(891, 12)
```

Nilai ini menunjukkan bahwa data Titanic memiliki 891 baris dan 12 kolom.

Pengecekan Tipe Data

```
df.info()
```
[10] ✓ 0.0s

Perhatikan hasil dari perintah yang telah dijalankan.

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    str
 4   Sex          891 non-null    str
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    str
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    str
 11  Embarked     889 non-null    str
dtypes: float64(2), int64(5), str(5)
memory usage: 118.9 KB
```

Dari df.info() kita dapat mengetahui jumlah data untuk setiap kolom. Selain itu, kita mengetahui bahwa,

- "Age" berjenis float64
- "Cabin" berjenis object
- "Embarked" berjenis object

Pada Pandas, object sering kali mewaliki tipe data string. Kita harus cermat dalam hal ini. Hal ini berkaitan dengan teknik imputasi yang akan kita gunakan nantinya.

Selanjutnya, untuk lebih mengetahui jumlah data yang hilang untuk setiap kolom, gunakan perintah,

```
[11]  df.isnull().sum()
      ✓ 0.0s
```

Didapatkan informasi seperti berikut,

```
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64
```

Dari informasi ini, kita dapat mengetahui data yang hilang sejumlah,

- 177 data "Age"
- 687 data "Cabin"
- 2 data "Embarked"

Informasi ini akan kita gunakan pada praktikum selanjutnya.

Ringkasan Nilai Statistik
Untuk mendapatkan ringkasan nilai statistik, gunakan,

```
[12]  df.describe()
      ✓ 0.0s
```

Anda akan mendapatkan informasi penting seperti,

1. Jumlah data
2. Nilai mean
3. Nilai simpangan baku (standard deviation)
4. Nilai min dan max
5. Nilai Q1 (25%), Q2 (50%), dan Q3 (75%).

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

Langkah 3 - Inspeksi Visual

Inspeksi visual dilakukan untuk mengetahui pola data secara visual. Dengan cara ini kita dapat menentukan langkah pra pengolahan data dengan lebih tepat. Inspeksi visual yang akan dilakukan adalah,

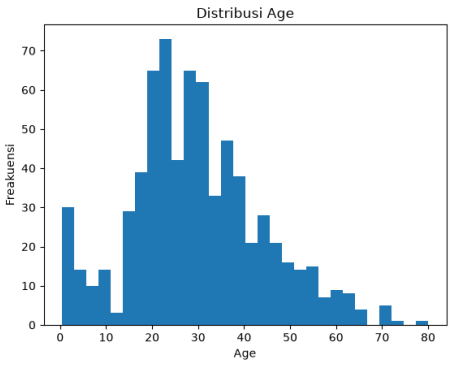
1. Inspeksi distribusi data melalui histogram
2. Inspeksi outlier melalui boxplot
3. Inspeksi korelasi antar variabel dengan heatmap dan scatterplot

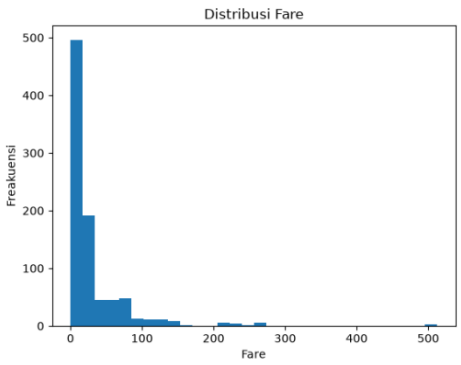
Inspeksi Distribusi Data
Lakukan inspeksi distribusi data terhadap variabel Age dan Fare.

```
num_cols = ['Age','Fare']
for col in num_cols:
    plt.figure()
    plt.hist(df[col], bins=30)
    plt.title(f'Distribusi {col}')
    plt.xlabel(col); plt.ylabel('Freakuensi')
    plt.show()
```

[13] ✓ 2.9s

Anda akan mendapatkan histogram untuk Age dan Fare seperti berikut,





Secara sekilas, kita dapat menyimpulkan terjadi fenomena skew pada distribusi Age dan Fare.

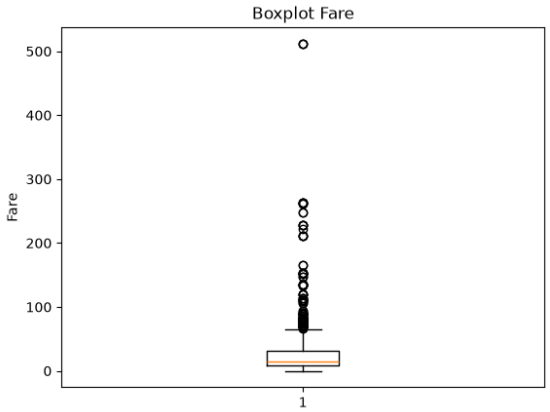
Inspeksi Outlier

Inspeksi terhadap nilai-nilai outlier juga penting dalam proses EDA. Outlier dapat mengakibatkan pergeseran distribusi data yang dapat berdampak pada proses pelatihan model. Pada praktikum ini kita akan melakukan pengecekan outlier pada 1 variabel saja, yaitu Fare. Visualisasi yang paling mudah untuk digunakan dalam mendeteksi outlier adalah dengan menggunakan boxplot.

```
plt.figure()
plt.boxplot(df['Fare'].dropna(), vert=True)
plt.title('Boxplot Fare')
plt.ylabel('Fare')
plt.show()
```

[14] ✓ 0.2s

Anda akan mendapatkan hasil seperti berikut,



Boxplot hanya dapat digunakan untuk data kuantitatif.

Inspeksi Komposisi Variabel Kualitatif

Dengan menggunakan boxplot, kita hanya mampu melakukan inspeksi pada variabel kuantitatif. Namun terkadang kita juga perlu melakukan inspeksi pada variabel kualitatif. Salah satu inspeksi yang dapat kita lakukan pada variabel kualitatif adalah mencari tahu jumlah data tiap kategorinya. Kita dapat menggunakan barchart dalam konteks ini.

Sebagai contoh, kita akan inspeksi variabel `Survived`, `Pclass`, `Sex`, `Embarked`.

```
cat_cols = ['Survived', 'Pclass', 'Sex', 'Embarked']

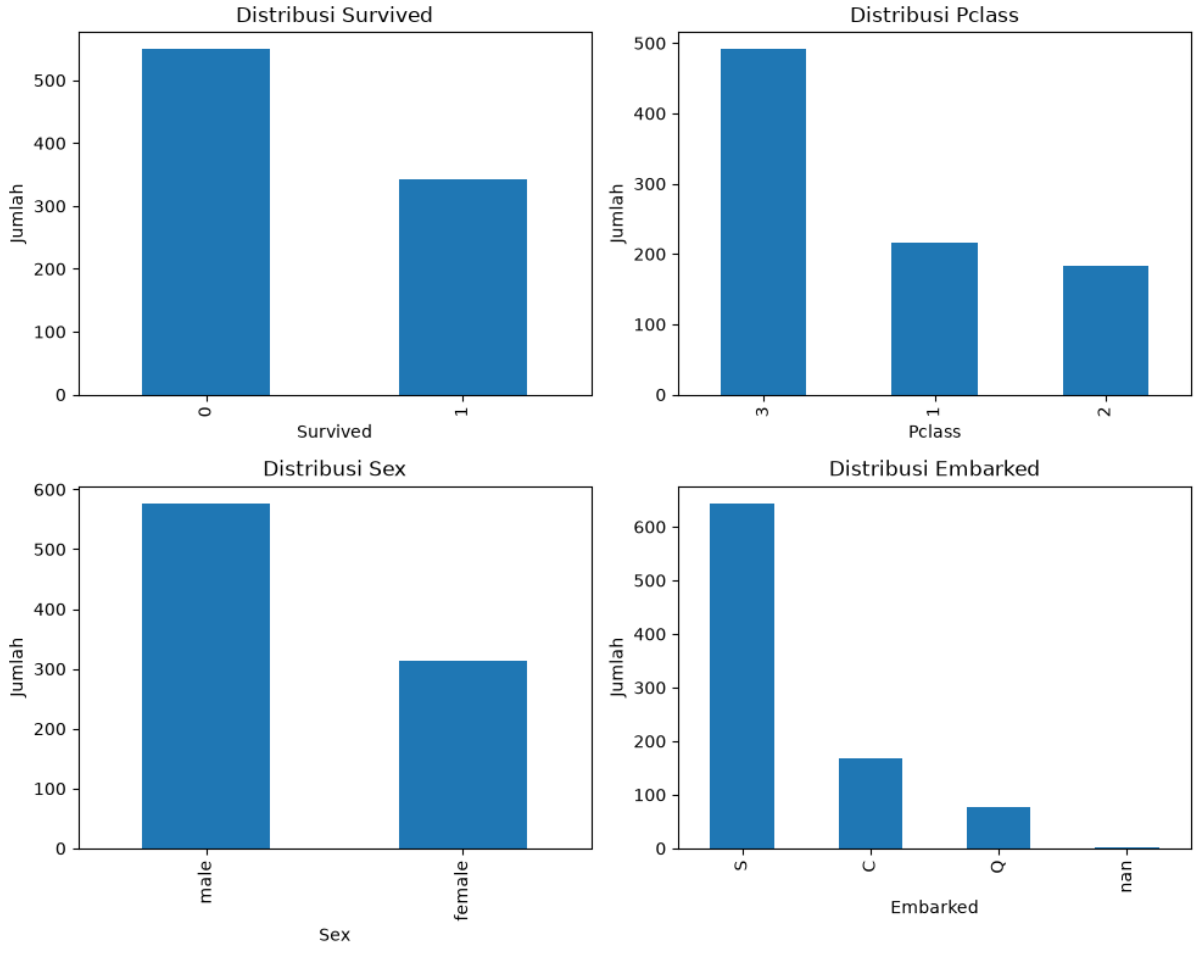
fig, axes = plt.subplots(2, 2, figsize=(10, 8))
axes = axes.flatten()

for i , col in enumerate(cat_cols):
    df[col].value_counts(dropna=False).plot(kind='bar', ax=axes[i])
    axes[i].set_title(f'Distribusi {col}')
    axes[i].set_xlabel(col)
    axes[i].set_ylabel('Jumlah')

plt.tight_layout()
plt.show()
```

```
[15]  ✓  1.7s
```

Hasilnya,



Informasi yang kita dapatkan dari sini adalah, apakah jumla tiap kategorinya berimbang atau bahkan ada yang bernilai kosong (NaN).

Inspeksi Korelasi

Nilai korelasi digunakan untuk mengetahui apakah terdapat hubungan saling mempengaruhi antar variabel. Nilai ini cukup penting untuk melihat hubungan antara variabel independen dengan dependennya. Jika nilai korelasi tidak cukup, maka kita perlu melakukan tindakan lanjutan salah satunya dengan tidak menggunakan variabel independen tersebut dalam proses selanjutnya.

Dalam praktikum ini kita akan menggunakan 2 visualisasi untuk melakukan inspeksi korelasi, yaitu heatmap dan scatterplot.

Heatmap

Kita akan mennggunakan library Seaborn pada visualisasi heatmap.

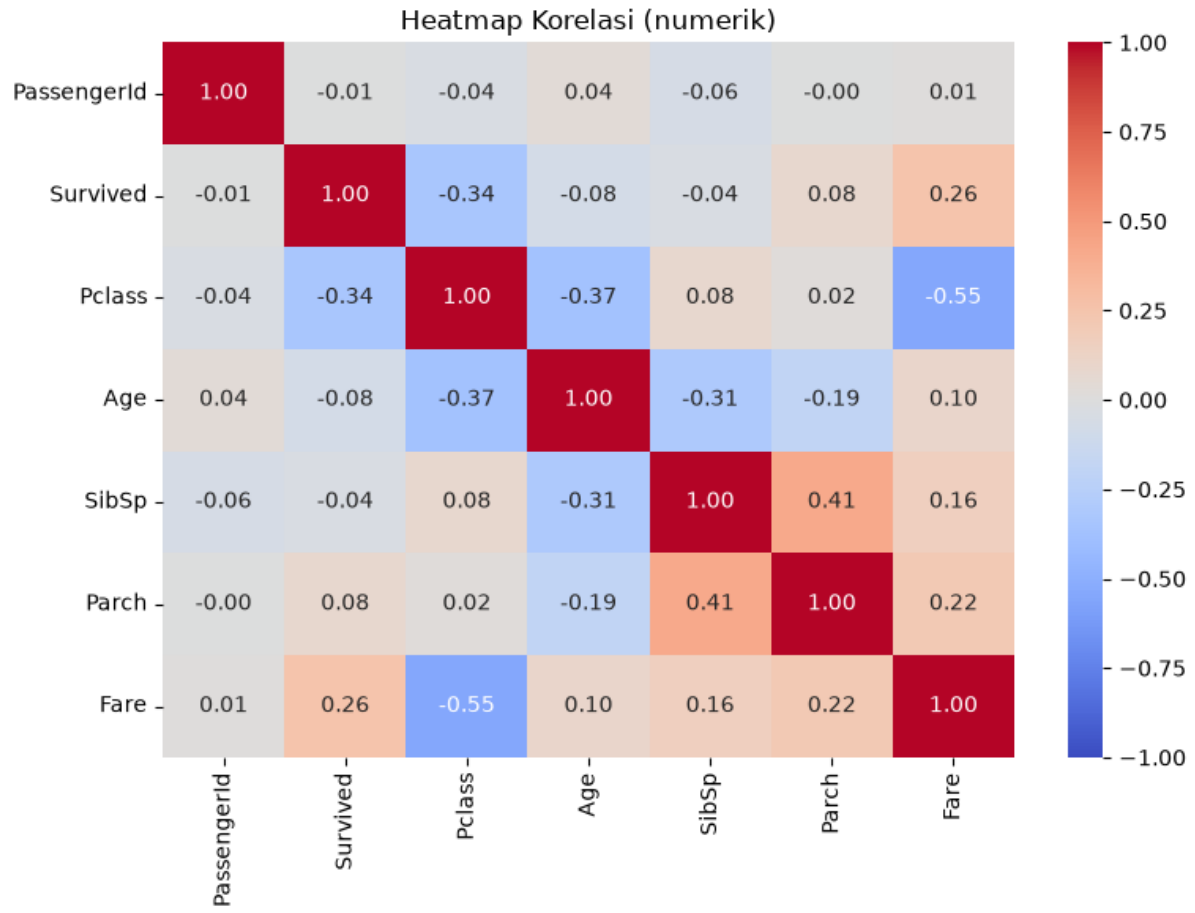
```
[17]  ✓ 0.9s
```

```python
import seaborn as sns # Import seaborn for enhanced visualization

num_only = df.select_dtypes(include=[np.number])
corr = num_only.corr(numeric_only=True)

plt.figure(figsize=(8, 6)) # Increased figure size for better readability
sns.heatmap(corr, cmap='coolwarm', vmin=-1, vmax=1, annot=True, fmt=".2f") # Use seaborn.heatmap with annotation
plt.title('Heatmap Korelasi (numerik)')
plt.tight_layout()
plt.show()
```

Hasil dari analisis heatmap adalah sebagai berikut,



Secara ringkat kita juga dapat melakukan inspeksi korelasi dengan hasil data tabular. Sebagai contoh, variabel Survived adalah variabel dependen yang akan menjadi fokus. Gunakan perintah,

```
corr['Survived'].sort_values(ascending=False)
```

```
[18]  ✓ 0.0s
```

Maka Anda akan mendapatkan informasi nilai korelasi variabel lain terhadap Survived.

```
Survived       1.000000
Fare           0.257307
Parch          0.081629
PassengerId   -0.005007
SibSp         -0.035322
Age           -0.077221
Pclass        -0.338481
Name: Survived, dtype: float64
```

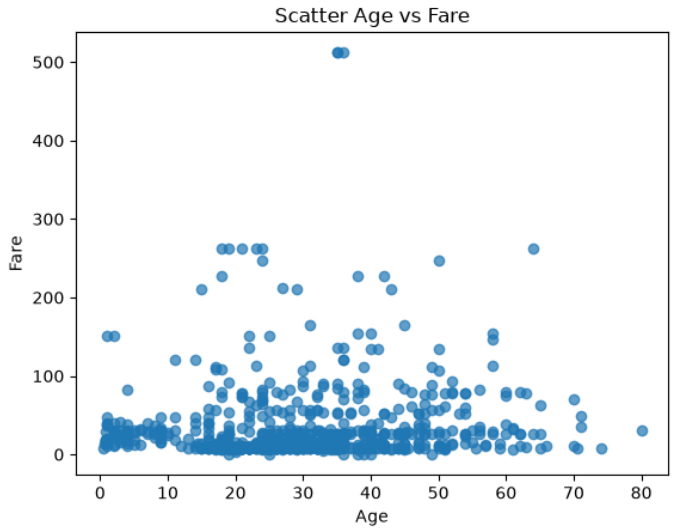
Scatter Plot

Scatter plot biasanya digunakan untuk mendapatkan informasi distribusi variabel dan juga dapat digunakan untuk mengetahui hubungan antar variabel. Hubungan ini dapat dijelaskan melalui pola-pola linier maupun non-linier.

Contoh, kita akan melakukan inspeksi pada variabel Age dan Fare.

```
plt.figure()
surv_map = {0:'No', 1:'Yes'}
plt.scatter(df['Age'], df['Fare'], alpha=0.7)
plt.xlabel('Age'); plt.ylabel('Fare')
plt.title('Scatter Age vs Fare')
plt.show()
```
[19] ✓ 0.5s

Anda akan mendapatkan hasil,



Berdasarkan hasil scatterplot, Age dan Fare tidak menunjukkan hubungan korelasi yang cukup kuat. Hal ini nampak pada setiap rentang usia memiliki harga tiket yang tidak jauh bervariasi.

Praktikum 2b
Pra Pengolahan Data - Data Terstruktur

Data Imputation

Pada praktikum ini kita akan mencoba melakukan data imputation terhadap data yang hilang (*missing value*). Dataset yang digunakan adalah dataset Titanic.

Langkah 0 - Persiapan

Lakukan import library yang dibutuhkan.

```
import pandas as pd
import numpy as np
```
[1] ✓ 13.0s

Langkah 1 - Memuat Data

Lakukan loading data Titanic pada notebook.

```
df = pd.read_csv('Titanic-Dataset.csv')

#Tampilkan 5 data teratas
df.head()
```
[2] ✓ 0.1s

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

Langkah 2 - Inspeksi Data

Anda dapat melakukan inspeksi singkat pada data untuk mendapatkan informasi ringkas yang diperlukan seperti pada Langkah 2 - Inspeksi Data.

Langkah 3 - Data Imputation

Pada langkah ini kita akan melakukan imputasi terdapat data yang hilang pada "Age", "Cabin", dan "Embarked".
Strategi yang akan kita gunakan adalah,

- "Age" → Dikarenakan "Age" adalah data nominal, maka kita akan menggunakan strategi mean.
- "Cabin" → "Cabin" merupakan informasi terkait dengan nomor kabin penumpang. Disini kita akan berasumsi bahwa, seluruh penumpang yang tidak memiliki nomor kabin, merupakan penumpang yang tinggal di dek-dek kapal. Sehingga kita akan mensubtitusi data yang hilang dengan informasi "DECK".
- "Embarked" → "Embarked" merupakan informasi lokasi embarkasi penumpang. Hanya ada 2 data yang hilang disini. Dikarenakan data merupakan data nominal, maka kita dapat menggunakan modus (mode) untuk mensubtitusi data yang hilang.

```
# Age - mean
df['Age'].fillna(value=df['Age'].mean(), inplace=True)

# Cabin - "DECK"
df['Cabin'].fillna(value="DECK", inplace=True)

# Embarked - modus
df['Embarked'].fillna(value=df['Embarked'].mode()[0], inplace=True)
```
[8] ✓ 0.0s

Langkah 4 - Validasi Hasil

Lakukan pengecekan kembali dengan df.info(). Selanjutnya, kita juga dapat memastikan kembali apakah nilai yang diisikan sudah benar dengan melakukan pencekan data secara langsung. Hasil dari proses imputasi adalah sebagai berikut,

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    str
 4   Sex          891 non-null    str
 5   Age          891 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    str
 9   Fare         891 non-null    float64
 10  Cabin        891 non-null    str
 11  Embarked     891 non-null    str
dtypes: float64(2), int64(5), str(5)
memory usage: 121.4 KB
```

Praktikum 2c
(Sekilas) Seleksi Fitur, Encoding, dan Standardisasi

Seleksi Fitur

Pada tahap ini kita akan memilih variabel mana saja yang akan kita gunakan sebagai fitur. Pada bagian dasar teori, kita sudah mempelajari bahwa tidak selalu semua variabel dapat digunakan sebagai fitur. Oleh karena itu, kita dapat memilih fitur yang sekiranya penting untuk tujuan yang ingin kita capai dalam membuat model pembelajaran mesin. Kita akan menggunakan kasus yang sama, yaitu kita kita mengetahui ciri-ciri penumpang kapal Titanic yang selamat. Sehingga, fitur yang kita gunakan adalah,

1. "**Survived**"
2. "**Pclass**"
3. "**Sex**"
4. "**Age**"
5. "**Cabin**"

Kita akan memisahkan variabel-variabel tersebut dari variabel yang lain.

Data yang akan kita gunakan adalah data Titanic yang sudah diperbaiki dengan proses imputasi.

Langkah 1 - Load Data

```
dpath = 'Titanic-Dataset-fixed.csv'
df = pd.read_csv(dpath)
df.head()
```

✓ 0.1s

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	DECK	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	DECK	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	DECK	S

Langkah 2 - Slice Data

Pilih variabel yang kita anggap sebagai fitur

```
df = df[['Survived', 'Pclass', 'Age', 'Sex', 'Cabin']]
df.head()
```

[7] ✓ 0.0s

Hasilnya

	Survived	Pclass	Age	Sex	Cabin
0	0	3	22.0	male	DECK
1	1	1	38.0	female	C85
2	1	3	26.0	female	DECK
3	1	1	35.0	female	C123
4	0	3	35.0	male	DECK

Langkah 3 - Encoding

Kita akan melakukan encoding pada data-data kategorikal, yaitu "Sex" dan "Cabin". Langkah ini perlu dilakukan karena algoritma pembelajaran mesin melakukan proses kalkulasi secara matematis. Strategi yang kita gunakan adalah **Label Encoding**. Label Encoding serupa dengan Ordinal Encoding, bedanya hanya pada proses pengurutan. Label Encoding tidak mengurutkan data terlebih dahulu.

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder() # membuat objek dari LabelEndcoder
df['Sex'] = le.fit_transform(df['Sex']) # proses encoding
df['Cabin'] = le.fit_transform(df['Cabin']) # proses encoding
```

[9] ✓ 0.1s

Langkah 4 - Verifikasi Hasil

Cek kembali dengan df.head(). Hasilnya akan seperti pada Gambar dibawah ini.

```
df.head()
```

[10] ✓ 0.0s

	Survived	Pclass	Age	Sex	Cabin
0	0	3	22.0	1	115
1	1	1	38.0	0	81
2	1	3	26.0	0	115
3	1	1	35.0	0	55
4	0	3	35.0	1	115

Langkah 5 - Standarisasi

Pada tahapan ini kita akan melakukan standardisasi pada "Age". **Hal ini sebetulnya tidak terlalu diperlukan karena nilai pada "Age" tidak terlalu jauh.** Tujuan dari langkah ini adalah untuk demonstrasi proses standardisasi.

```
from sklearn.preprocessing import StandardScaler

std = StandardScaler()
df['Age'] = std.fit_transform(df[['Age']])
```

[12] ✓ 0.0s

Langkah 6 - Verifikasi Hasil Standarisasi

Cek kembali dengan df.head(). Hasilnya akan seperti pada Gambar dibawah ini.

```
df.head()
```

[13] ✓ 0.0s

	Survived	Pclass	Age	Sex	Cabin
0	0	3	-0.592481	1	115
1	1	1	0.638789	0	81
2	1	3	-0.284663	0	115
3	1	1	0.407926	0	55
4	0	3	0.407926	1	115

Praktikum 2d
Pra Pengolahan Data - Data Tidak Terstruktur

Langkah 0 - Persiapan

Lakukan instalasi modul OpenCV pada perangkat komputasi.

```
!pip install -q opencv-python matplotlib
```
[14] ✓ 14.7s

Kemudian, import library yang dibutuhkan pada notebook.

```
import cv2
import matplotlib.pyplot as plt
```
[16] ✓ 2.7s

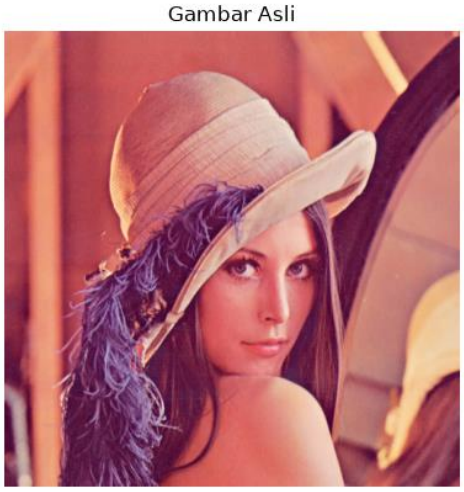
Langkah 1 - Load Gambar

Load gambar menggunakan OpenCV dan ubah colorspace menjadi RGB. Hal ini dikarenakan secara default, OpenCV akan membaca gambar dalam format BGR.

```
# Membaca gambar (BGR default di OpenCV)
img = cv2.imread('Lenna.png')

# Konversi BGR > RGB (agar warnanya tampil benar di matplotlib)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Tampilkan
plt.imshow(img_rgb)
plt.title("Gambar Asli")
plt.axis("off")
plt.show()
```
[17] ✓ 2.0s



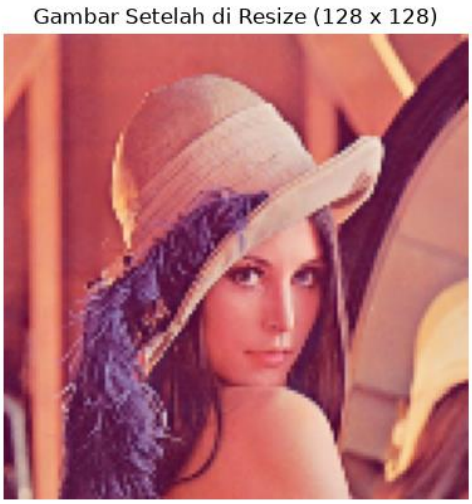
Langkah 2 - Resize Gambar

Selanjutnya, ubah ukuran gambar menjadi 128x128px.

```
# Resize gambar
img_resized = cv2.resize(img_rgb, (128, 128))

plt.imshow(img_resized)
plt.title("Gambar Setelah di Resize (128 x 128)")
plt.axis("off")
plt.show()
```
[18] ✓ 0.7s

Jika diperhatikan, akan terdapat perbedaan pada ketajaman gambar.



Langkah 3 - Ubah ke Grayscale

Selanjutnya kita akan mengubah warna citra ke dalam bentuk grayscale.

```
# Konversi Gambar ke Grayscale
img_gray = cv2.cvtColor(img_resized, cv2.COLOR_BGR2GRAY)

plt.imshow(img_gray, cmap="gray")
plt.title("Gambar Setelah di Grayscale")
plt.axis("off")
plt.show()
```
[19] ✓ 0.1s

Hasilnya sebagai berikut,



Dalam praktiknya, tentunya tidak hanya proses resize dan grayscale yang akan dilakukan. Proses pra pengolahan data pada data tidak terstruktur bergantung dengan tujuan pembuatan model.

TUGAS PRAKTIKUM

Pada tugas pratikum ini Anda akan menggunakan data "Wisconsin Breast Cancer". Data tersebut terdiri dari 569 data yang digunakan untuk mendiagnonis jenis kanker **Malignant (M) dan Benign (B)**. Tugas Anda adalah,

1. **Pisahkan antara variabel yang dapat digunakan dan variabel yang tidak dapat digunakan.**
   Jawab :
   Load data:

```
import pandas as pd

dpath = '../Data Praktikum/wbc.csv'
df = pd.read_csv(dpath)
df.head()
```

Hasil:

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	...	texture_worst
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	...	17.33
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	...	23.41
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	...	25.53
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	...	26.50
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	...	16.67

5 rows × 33 columns

perimeter_worst	area_worst	smoothness_worst	compactness_worst	concavity_worst	concave points_worst	symmetry_worst	fractal_dimension_worst	Unnamed: 32
184.60	2019.0	0.1622	0.6656	0.7119	0.2654	0.4601	0.11890	NaN
158.80	1956.0	0.1238	0.1866	0.2416	0.1860	0.2750	0.08902	NaN
152.50	1709.0	0.1444	0.4245	0.4504	0.2430	0.3613	0.08758	NaN
98.87	567.7	0.2098	0.8663	0.6869	0.2575	0.6638	0.17300	NaN
152.20	1575.0	0.1374	0.2050	0.4000	0.1625	0.2364	0.07678	NaN

Pisah variabel yang diguanakn dan tidak dapat digunakan

```
df_simpan = df['diagnosis']

df_drop = df.drop(columns=['id', 'diagnosis', 'Unnamed: 32'])
df_drop.head()
```

Disini saya menyimpan variabel diagnonis untuk proses encoding pada kolom diagnosis dan menyimpannya pada variabel ***df_simpan.***
Lalu dibawahnya saya menghapus 3 variabel, yakni id, diagnosis, dan Unnamed: 32 karena variabel tersebut tidak digunakan dan menyimpannya pada variabel ***df_drop***.
Setelahnya saya menampilkan data dengan 3 variabel yang sudah di hapus tersebut menggunakan ***df_drop.head()***

Hasil:

	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	symmetry_mean	fractal_dimension_mean
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883

5 rows × 30 columns

radius_worst	texture_worst	perimeter_worst	area_worst	smoothness_worst	compactness_worst	concavity_worst	concave points_worst	symmetry_worst	fractal_dimension_worst
25.38	17.33	184.60	2019.0	0.1622	0.6656	0.7119	0.2654	0.4601	0.11890
24.99	23.41	158.80	1956.0	0.1238	0.1866	0.2416	0.1860	0.2750	0.08902
23.57	25.53	152.50	1709.0	0.1444	0.4245	0.4504	0.2430	0.3613	0.08758
14.91	26.50	98.87	567.7	0.2098	0.8663	0.6869	0.2575	0.6638	0.17300
22.54	16.67	152.20	1575.0	0.1374	0.2050	0.4000	0.1625	0.2364	0.07678

2. **Lakukan proses encoding pada kolom "diagnosis".**

Jawab:

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df_simpan_encoded = le.fit_transform(df['diagnosis'])
```
✓ 0.0s

Import library LabelEncoder dari sklearn.preprocessing untuk proses Encoding pada kolom ***"diagnosis".***

Selanjutnya saya membuat objek dari LabelEncoder dengan nama ***le***.

Selanjutnya saya membuat variabel ***df_simpan_encoded*** untuk menyimpan hasil dari proses encoding pada kolom ***"diagnosis".***

Hasil Encoding:

```
df_simpan_encoded[:5]
```
✓ 0.0s

```
array([1, 1, 1, 1, 1])
```

Disini saya tidak menggunakan ***df_simpan_encoded.head()*** karena numpy array menggunakan slicing bukan method ***.head().***

3. **Lakukan proses standardisasi pada semua kolom yang memiliki nilai numerik.**

Jawab:

```
from sklearn.preprocessing import StandardScaler

std = StandardScaler()
df_drop_std = std.fit_transform(df_drop)
```
✓ 0.0s

Import library StandardScaler dari sklearn.preprocessing untuk proses standarisasi pada kolom ***"numerik"***

Selanjutnya saya membuat objek dari StandardScaler dengan nama ***std.***

Selanjutnya saya membuat variabel ***df_drop_std*** untuk menyimpan hasil dari proses standarisasi pada kolom numerik. *(Karena pada variabel df_drop semua kolom berisi numerik maka akan langung dilakukan proses standarisasi)

Hasil Standarisasi:

```
df_drop_std[:3, :30]
```
✓ 0.0s

```
array([[ 1.09706398e+00, -2.07333501e+00,  1.26993369e+00,
         9.84374905e-01,  1.56846633e+00,  3.28351467e+00,
         2.65287398e+00,  2.53247522e+00,  2.21751501e+00,
         2.25574689e+00,  2.48973393e+00, -5.65265059e-01,
         2.83303087e+00,  2.48757756e+00, -2.14001647e-01,
         1.31686157e+00,  7.24026158e-01,  6.60819941e-01,
         1.14875667e+00,  9.07083081e-01,  1.88668963e+00,
        -1.35929347e+00,  2.30360062e+00,  2.00123749e+00,
         1.30768627e+00,  2.61666502e+00,  2.10952635e+00,
         2.29607613e+00,  2.75062224e+00,  1.93701461e+00],
       [ 1.82982061e+00, -3.53632408e-01,  1.68595471e+00,
         1.90870825e+00, -8.26962447e-01, -4.87071673e-01,
        -2.38458552e-02,  5.48144156e-01,  1.39236330e-03,
        -8.68652457e-01,  4.99254601e-01, -8.76243603e-01,
         2.63326966e-01,  7.42401948e-01, -6.05350847e-01,
        -6.92926270e-01, -4.40780058e-01,  2.60162067e-01,
        -8.05450380e-01, -9.94437403e-02,  1.80592744e+00,
        -3.69203222e-01,  1.53512599e+00,  1.89048899e+00,
        -3.75611957e-01, -4.30444219e-01, -1.46748968e-01,
         1.08708430e+00, -2.43889668e-01,  2.81189987e-01],
       [ 1.57988811e+00,  4.56186952e-01,  1.56650313e+00,
         1.55888363e+00,  9.42210440e-01,  1.05292554e+00,
         1.36347845e+00,  2.03723076e+00,  9.39684817e-01,
        -3.98007910e-01,  1.22867595e+00, -7.80083377e-01,
         8.50928301e-01,  1.18133606e+00, -2.97005012e-01,
         8.14973504e-01,  2.13076435e-01,  1.42482747e+00,
         2.37035535e-01,  2.93559404e-01,  1.51187025e+00,
        -2.39743838e-02,  1.34747521e+00,  1.45628455e+00,
         5.27407405e-01,  1.08293217e+00,  8.54973944e-01,
         1.95500035e+00,  1.15225500e+00,  2.01391209e-01]])
```

Disini saya tidak menggunakan ***df_drop_std.head()*** karena numpy array menggunakan slicing bukan method ***.head().*** Saya juga menampilkan data yang sudah di Standarisasi pada 3 baris 30 kolom.